



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

GAZIPUR-1704, DHAKA, BANGLADESH

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

PROJECT REPORT

EEE 4518

ELECTRICAL AND ELECTRONIC WORKSHOP

***PROJECT TITLE:* Automated Script Checker**

SUBMITTED BY:

Team Name: Bootloader

- Jubair Bin Masud **220021133**
- Shahriar Fahim **220021326**
- Mohammad Asiful Islam **220021353**

DATE OF SUBMISSION: 04.03.2025

Table of Contents

1. Introduction	3
2. Objectives	3
3. Required Components	3
4. Circuit Diagram	4
4.1 Explanation	4
5. Features	5
6. Working Principle	5
6.1 Flowchart	6
6.2 Explanation	6
7. Code	7
8. Hardware Demonstration	9
9. Problems Faced	10
10. Conclusion	10
11. References	10
12. Contributions	10
12.1 Contribution of team member 1	11
12.2 Contribution of team member 2	
12.3 Contribution of team member 3	11
13. CO PO covered	12

1. Introduction

The Automated Script Checker project aims to automate the grading of handwritten answer sheets using a scanning camera and a robotic arm. The system detects specific symbols (such as squares, circles, and triangles) on the paper using computer vision (OpenCV) to classify and compare them to a predefined answer key. If any errors are found, the system calculates the exact location of the incorrect symbol and uses a robotic arm to either correct or mark the mistake. The goal is to replace manual grading, saving time and ensuring consistent, unbiased evaluations yet providing traditional, real-world actions.

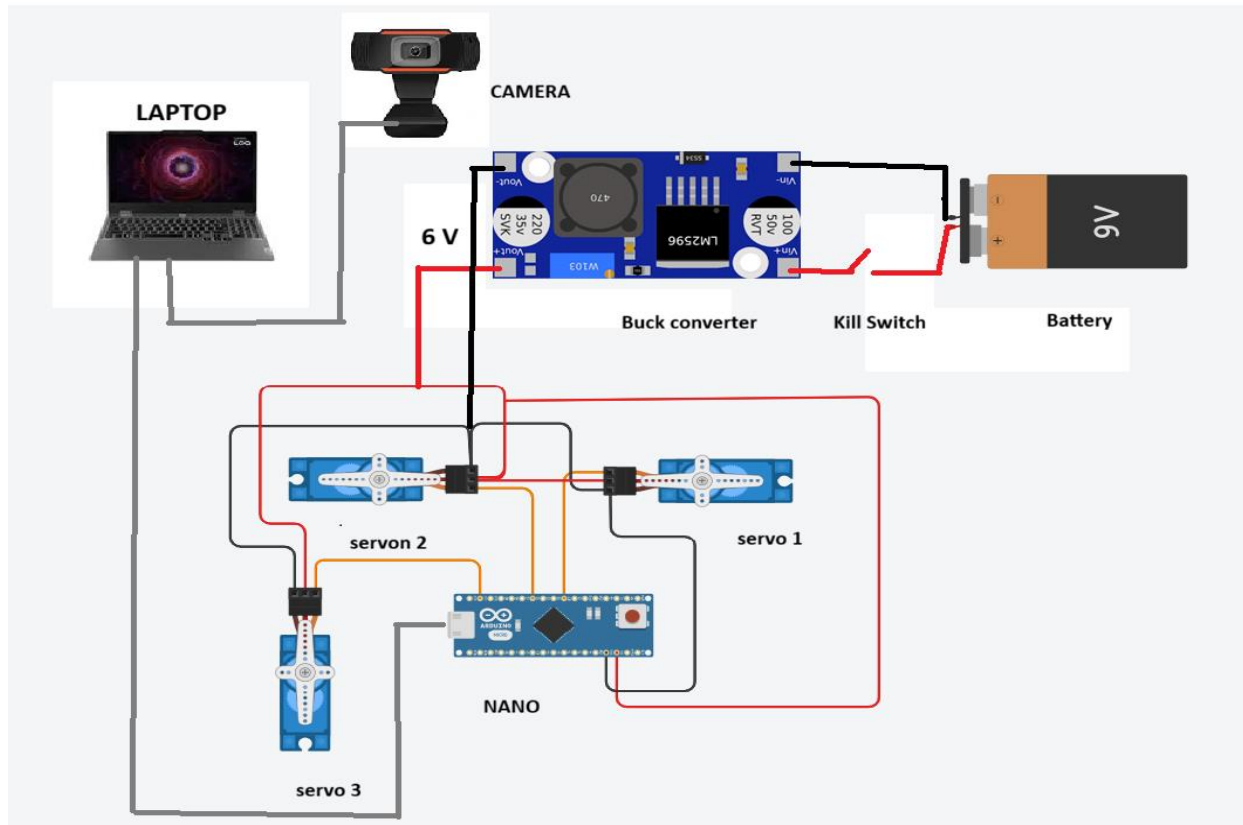
2. Objectives

- Automate the grading process: Detect and evaluate symbols drawn on answer sheets without human intervention.
- Develop a computer vision system: Use OpenCV to capture and process images of answer sheets to classify and compare shapes (square, circle, triangle).
- Design a robotic arm: Control a 3-degree-of-freedom robotic arm to mark or correct detected errors.
- Real-time feedback and results: Provide immediate updates on the grading accuracy and mark the errors directly on the paper.

3. Required Components

- Microcontroller: An Arduino is used to control the robotic arm and receive instructions from the computer system.
- Servo Motors: To control the movements of the robotic arm.
- Webcam: For capturing images of the answer sheet.
- Robotic Arm: A SCARA-type robotic arm capable of precise movement to mark or correct symbols.
- Power Supply: To provide sufficient power to the microcontroller and servo motors.
- Frame Materials: PVC or other materials are used for building the physical structure to support the arm and camera.

4. Circuit Diagram



4.1 Explanation

- **Microcontroller:** Receives processed data from the computer (such as detected positions of incorrect symbols) and commands the robotic arm accordingly.
- **Servo Motors:** These motors move the joints of the robotic arm to perform tasks such as marking or drawing on the answer sheet.
- **Webcam:** The camera captures the image of the answer sheet. OpenCV processes this image to detect shapes, and the coordinates of the symbols are used for error marking.
- **Power Supply:** Ensures that all components, especially the motors, operate smoothly and efficiently.
- **Buck converter:** Provides suitable power to microcontroller and Servo motors to get desired torque output

5. Features

- Automatic Error Detection: The system detects shapes and compares them to the correct answer key, marking or correcting the mistakes automatically.
- Robotic Arm: The arm can draw, mark, and correct errors on the paper, performing a physical task based on the computer's evaluation.
- Modular Design: The system is divided into distinct modules—vision system, coordinate transformation, robotic arm control—making it easy to develop and test each part separately.
- Real-time Operation: The system works in real-time, capturing the image, processing it, and delivering immediate feedback.

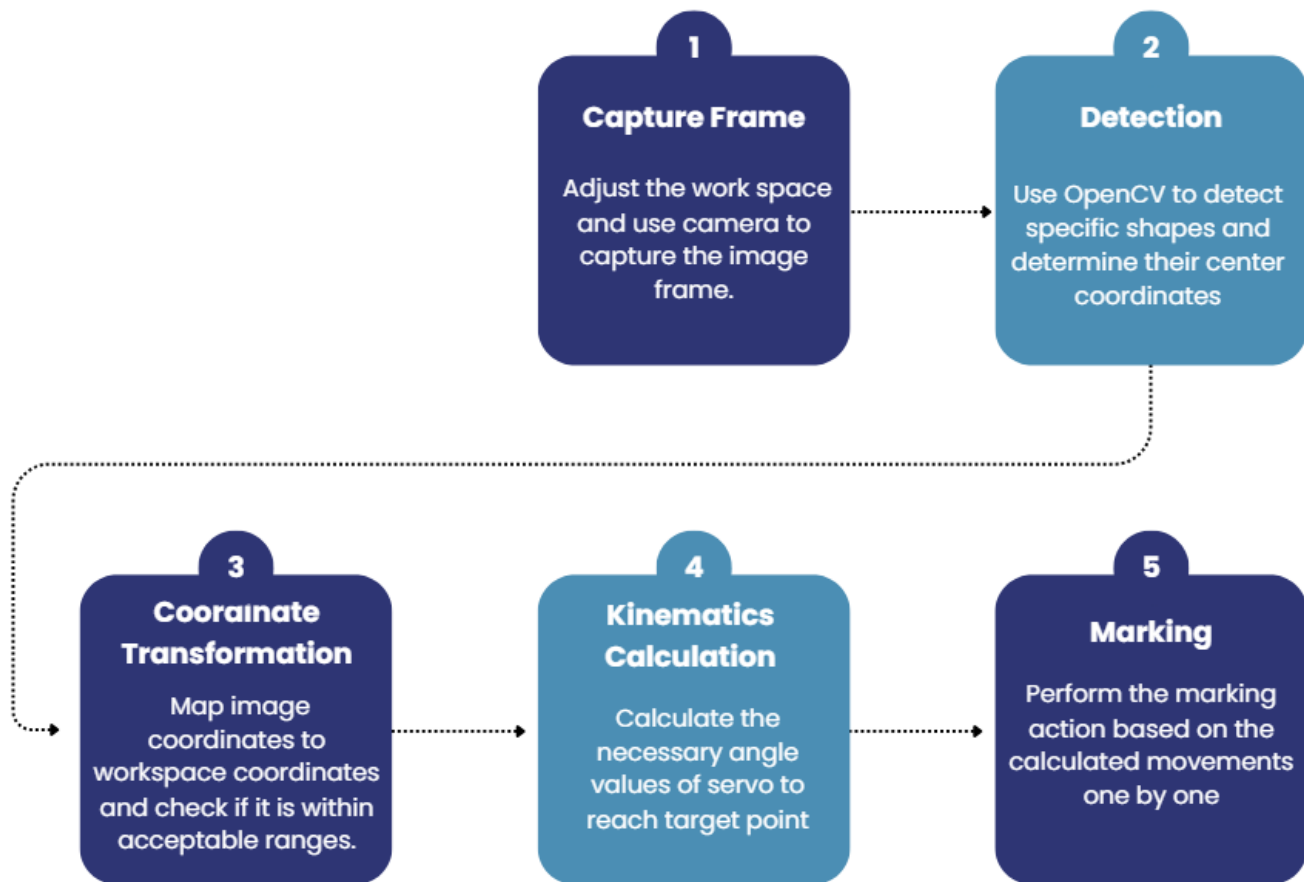
Potential Features

- Correct shape plotting
- Obtained marks potting
- Text detection and spelling check
- AI based evaluation

6. Working Principle

1. Image Acquisition: The webcam captures an image of the answer sheet.
2. Object Detection: OpenCV is used to process the image, detecting and identifying shapes on the paper.
3. Coordinate Transformation: The detected object coordinates are mapped to real-world coordinates that the robotic arm can understand and use.
4. Inverse Kinematics: The calculated coordinates are fed into the inverse kinematics algorithm to determine the exact movement required by the robotic arm.
5. Marking and Correction: The robotic arm moves to the correct position and marks or draws on the answer sheet based on the identified errors.

6.1 Flowchart



6.2 Explanation

The flowchart visualizes how the system operates from capturing the image to processing it and performing actions based on the detected errors. It starts with the image capture, followed by symbol detection using OpenCV. The system then transforms the detected 2D pixel coordinates into 2D real space coordinates, calculates the necessary arm movements using inverse kinematics, and finally marks or corrects the mistakes on the paper.

7. Code

Python

1. User Mode & Configuration:
 - Specifies detection mode (SQUARE, TRIANGLE, CIRCLE) and configures settings for image capture and serial communication.
2. Serial Communication:
 - Initializes serial communication with Arduino and sends data using `send_line()`.
3. Coordinate Transformation:
 - `py_to_arduino_coords()`: Converts pixel coordinates to Arduino-compatible ones, adjusting for perspective distortion and skew.
4. Shape Detection:
 - `is_square_like()`, `is_triangle()`, `is_circle()`: Detects square, triangle, and circle shapes based on contours and geometric properties.
5. Image Capture & Preprocessing:
 - Captures the image and applies preprocessing using `cv2.cvtColor()`, `cv2.bilateralFilter()`, and `cv2.adaptiveThreshold()`.
6. Contour Detection:
 - `cv2.findContours()`: Identifies potential shapes by finding contours in the processed image.
7. Shape Validation:
 - Iterates through contours, labels shapes as "OK" or "WRONG", and stores incorrect shape coordinates using `contour_center()` and `py_to_arduino_coords()`.
8. Arduino Communication:
 - Sends incorrect shape coordinates and the count of correct shapes to Arduino using `send_line()`.
9. Display Results:
 - Displays the processed image with labeled shapes and the overall detection score using `cv2.imshow()`.

Arduino

1. Servo Setup & Motion Control:

- Uses three servos: two for arm movement (servo1, servo2) and one for the pen (penServo).
- Inverse kinematics calculates the required servo angles to position the arm at specific coordinates.

2. Pen Control:

- penUp() and penDown() control the pen's position to start or stop drawing.

3. Inverse Kinematics:

- inverseKinematics() calculates angles for the servos using the law of cosines to move the arm to a given (x, y) position.

4. Smooth Motion:

- Interpolates servo angles for smooth movement, reducing jerky motions.

5. Drawing Shapes:

- **Circle Drawing:** Calculates points on a circle's perimeter and moves the arm to draw it.
- **Parallel Lines:** Draws multiple parallel lines with a specified length, spacing, and number.

6. Serial Communication:

- Listens for commands (BEGIN, END, START, C,x,y, CORRECT) to control drawing and store coordinates for shapes.

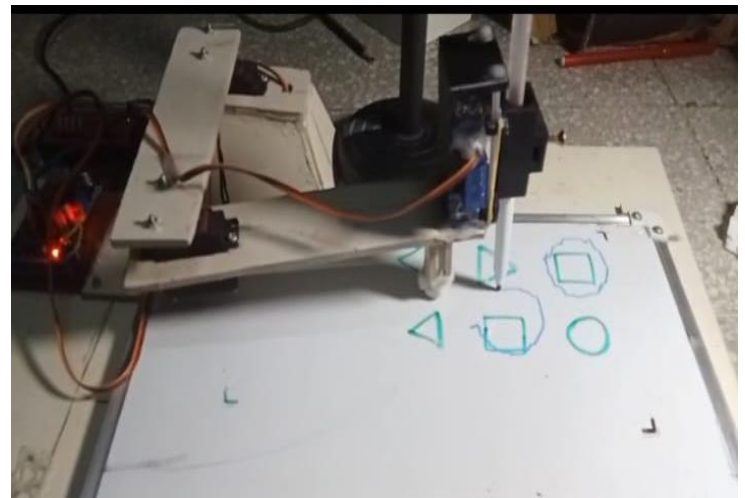
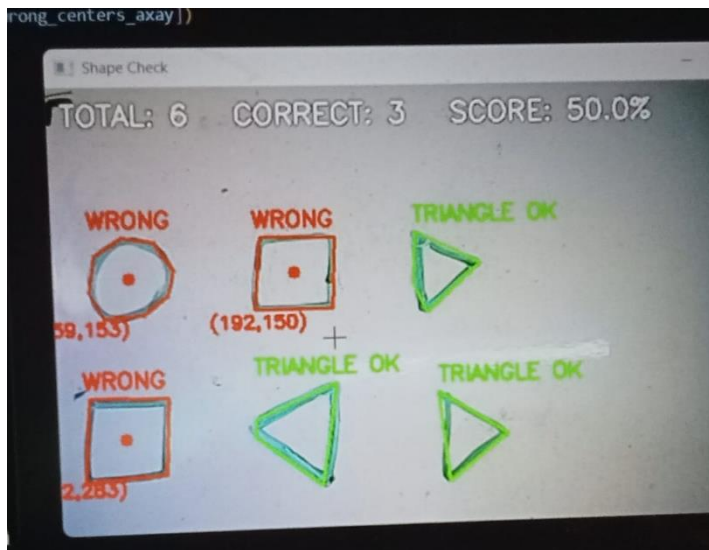
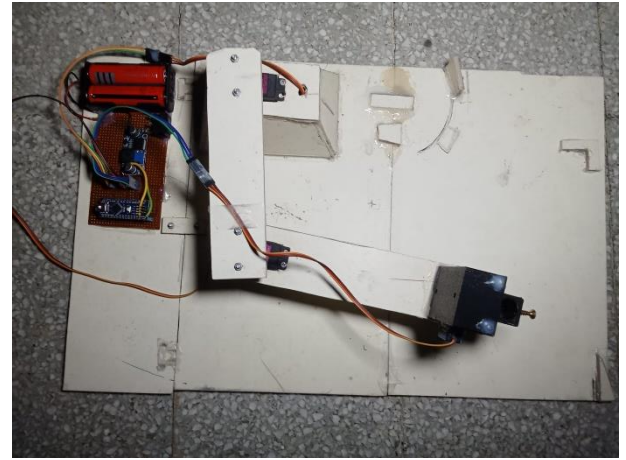
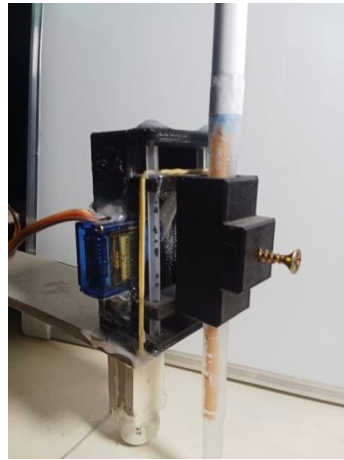
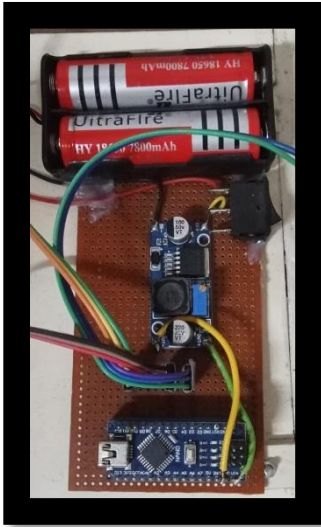
7. Main Logic:

- In setup(), servos are initialized, and the arm moves to a "home" position.
- In loop(), the arm draws shapes based on serial commands, then draws parallel lines to indicate the correct count.

8. Drawing Flow:

- The arm returns to home, draws circles based on received centers, then draws parallel lines, and returns home after completing the task.

8. Hardware Demonstration



9. Problems Faced

- Camera Calibration: Initially, the camera's field of view and distortion were issues that required adjustment.
- Symbol Detection Accuracy: Ensuring that the system detects the correct symbols required fine-tuning and rigorous image processing steps.
- Robotic Arm Precision: Calibrating the arm's movements for precise error correction involved careful tuning of the inverse kinematics algorithm.

10. Conclusion

- The Automated Script Checker has successfully combined computer vision and robotics to automate the grading process. The system detects symbols on answer sheets, compares them with the correct answer key, and marks errors using a robotic arm. Despite some challenges in calibration and synchronization, the project proves the viability of using automation for educational grading. Future work could involve Correct shape plotting, obtained marks potting, text detection and spelling check, AI based evaluation etc.

11. References

- OpenCV Documentation: <https://docs.opencv.org/>
- Arduino Servo Control: <https://www.arduino.cc/en/Reference/Servo>
- CNC Pen Lifting: <https://www.instructables.com/CNC-Pen-Lift-1/>

12. Contributions

12.1 Contribution of Jubair (220021133)

- Project vision and planning
- Robotic arm design
- Inverse Kinematics
- Mechanical system
- Electrical system
- Hardware-software coordination

12.2 Contribution of Fahim (220021326)

- Assistance in OpenCV implementation
- Training of AI model (YOLO) to detect shapes
- Debugging proper image processing
- Text based implementation OCR (potential feature)

12.3 Contribution of Asif (220021353)

- Assistance in design and construction of the structure
- Interfacing python with Arduino-pen lifting mechanism
- UI development (potential feature)
- Text based implementation using OCR (potential feature)

CO/PO Covered	Justification	
CO1	Analyze and evaluate the work based on object detection and robotic control, focusing on data synthesis and investigation.	✓
CO2	Design and develop the workflow for detecting and correcting errors using the robotic arm.	✓
CO3	Use modern tools like OpenCV and microcontrollers for solving real-world problems in automation.	✓
CO4	Effectively communicate the project's design and implementation process in both oral and written forms.	✓
PO1	(Engineering knowledge): Apply knowledge of mathematics and engineering fundamentals like inverse kinematics and geometry.	✓
PO2	(Problem Analysis): Identifying, formulating, and analyzing complex issues related to the project's core tasks.	✓
PO3	(Design/Development of Solutions): Designing and developing solutions (robotic arm, vision system, kinematics) to meet the project objectives.	✓
PO5	(Modern Tool Usage): Using tools like OpenCV and Arduino to address real-world challenges.	✓